

EXPERIMENT NO. 7

Aim

To study the structure of Solidity programs, basic Solidity constructs, and the compilation and deployment environment, and to gain hands-on experience in creating and interacting with smart contracts using Remix IDE.

Theory

Solidity is a high-level programming language used to develop smart contracts on Ethereum-compatible blockchain platforms. A smart contract is a program stored on a blockchain that contains predefined rules and functions that can be executed by users or other contracts.

A Solidity program generally consists of a **license declaration, pragma statement, contract declaration, state variables, constructors, modifiers and functions.**

1. Primitive Data Types and Variables

Solidity provides several basic data types:

- **uint / int:** Used for unsigned and signed integers.
- **bool:** Represents **true** or **false**.
- **address:** Stores an Ethereum account or smart contract address.
- **string:** Stores textual data.
- **bytes:** Stores binary data.

Variables in Solidity can be classified as:

- **State variables:** Stored permanently on the blockchain.
- **Local variables:** Exist temporarily during function execution.
- **Global variables:** Predefined blockchain-related variables such as **msg.sender**, **msg.value** and **block.timestamp**.

2. Functions

Functions contain the executable logic of a smart contract.

A **pure** function does not read or modify the contract's state.

```
function add(uint a, uint b)
    public
    pure
```

```
    returns(uint)
{
    return a + b;
}
```

A **view** function can read state variables but cannot modify them.

```
function getValue()
    public
    view
    returns(uint)
{
    return value;
}
```

3. Function Inputs and Outputs

Functions can accept input parameters and return one or more values.

For example:

```
function multiply(uint a, uint b)
    public
    pure
    returns(uint)
{
    return a * b;
}
```

Here, **a** and **b** are inputs and the multiplication result is the output.

4. Visibility

Solidity provides four major visibility types:

- **public:** Can be accessed internally and externally.
- **private:** Can only be accessed within the same contract.
- **internal:** Can be accessed within the contract and derived contracts.
- **external:** Intended to be called from outside the contract.

5. Modifiers

Modifiers are reusable pieces of code that can restrict or modify the execution of functions. They are commonly used for access control.

Example:

```
modifier onlyOwner() {  
    require(msg.sender == owner, "Not owner");  
    _;  
}
```

A function using this modifier can only be executed when the specified condition is satisfied.

6. Constructors

A constructor is executed when a smart contract is deployed. It is generally used to initialize state variables.

Example:

```
constructor() {  
    owner = msg.sender;  
}
```

7. Control Flow

Solidity supports standard control-flow statements such as:

- `if-else`
- `for`
- `while`
- `do-while`

These statements are used for conditional and repeated execution of code. Loops must be used carefully in smart contracts because their execution consumes gas.

8. Data Structures

Solidity supports several data structures.

Arrays store ordered collections of values.

```
uint[] public numbers;
```

Mappings store key-value pairs.

```
mapping(address => uint) public balances;
```

Structs allow multiple related values to be grouped together.

```
struct Player {  
    string name;  
    uint score;
```

```
}
```

Enums define a fixed set of named values.

```
enum Status {  
    Pending,  
    Active,  
    Closed  
}
```

9. Data Locations

Solidity mainly uses three data locations:

- **storage:** Persistent data stored on the blockchain.
- **memory:** Temporary data that exists during function execution.
- **calldata:** Temporary and read-only data generally used for external function parameters.

10. Ether, Wei and Transactions

Ether (ETH) is the native currency of the Ethereum network.

The smallest unit of Ether is **Wei**:

1 Ether = 10^{18} Wei

Transactions are used to interact with smart contracts or transfer Ether.

A function that receives Ether generally needs to be declared **payable**.

11. Gas and Gas Price

Gas represents the computational work required to execute an Ethereum transaction or smart-contract operation.

The transaction cost is conceptually related to:

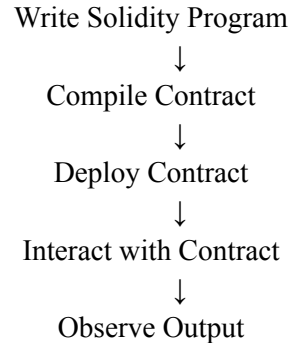
Transaction Fee = Gas Used × Gas Price

Gas encourages efficient smart-contract design because computationally expensive operations require more resources.

12. Remix IDE

Remix IDE is a browser-based development environment for writing, compiling, deploying and testing Solidity smart contracts.

The basic smart-contract development process is:



For this experiment, the **Remix VM** is used as the testing/deployment environment. The experiment requires exploring the Solidity Basics Course available through Remix LearnETH and completing its 19 assignments.

Procedure

Step 1: Open Remix IDE

Open the Remix IDE using:

<https://remix.ethereum.org/>

Navigate to the Solidity/LearnETH section and open the **Solidity Basics Course**.

Step 2: Explore the Solidity Basics Course

Study the concepts and examples provided in the LearnETH Solidity Basics Course.

The course contains **19 assignments/tutorials** that demonstrate different Solidity programming concepts.

Step 3: Open Tutorial 1

Open **Tutorial 1** from the Solidity Basics Course.

Study the provided Solidity smart contract and understand its variables and functions.

Step 4: Compile the Smart Contract

1. Open the **Solidity Compiler** panel in Remix IDE.
2. Select a compiler version compatible with the contract.
3. Compile the Solidity program.
4. Verify that there are no compilation errors.
5. Record the successful compilation output.

Step 5: Deploy the Smart Contract

1. Open **Deploy & Run Transactions**.
2. Select the **Remix VM** environment.
3. Select the appropriate account.
4. Select the compiled contract.
5. Click **Deploy**.
6. Verify that the contract appears under **Deployed Contracts**.

Step 6: Execute the **get** Function

After deployment, interact with the deployed contract.

Click the **get** function and observe the value returned by the contract.

Record the output.

Step 7: Execute the **increment** Function

Click the **increment** function.

This performs a state-changing operation on the smart contract.

After executing it, call the **get** function again and observe the updated value.

Step 8: Execute the **decrement** Function

Click the **decrement** function.

After the transaction is completed, execute the **get** function again and observe the updated value.

Step 9: Complete Tutorials 2–19

Continue with the remaining assignments in the Solidity Basics Course:

Tutorial 2
Tutorial 3
Tutorial 4
Tutorial 5
Tutorial 6
Tutorial 7
Tutorial 8
Tutorial 9
Tutorial 10
Tutorial 11
Tutorial 12

Tutorial 13
Tutorial 14
Tutorial 15
Tutorial 16
Tutorial 17
Tutorial 18
Tutorial 19

For each tutorial:

1. Open the assignment.
2. Study the Solidity program.
3. Compile the program.
4. Deploy the contract using Remix VM where required.
5. Execute the functions provided by the tutorial.
6. Observe and record the outputs.
7. Take the required screenshots for documentation.

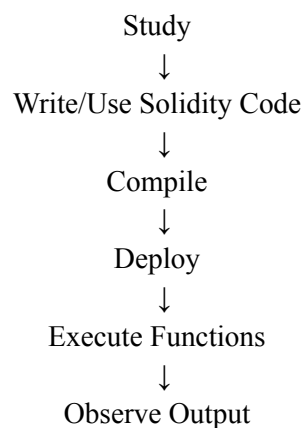
Step 10: Verify Smart Contract Interaction

For each deployed contract, interact with its available functions through the **Deployed Contracts** section.

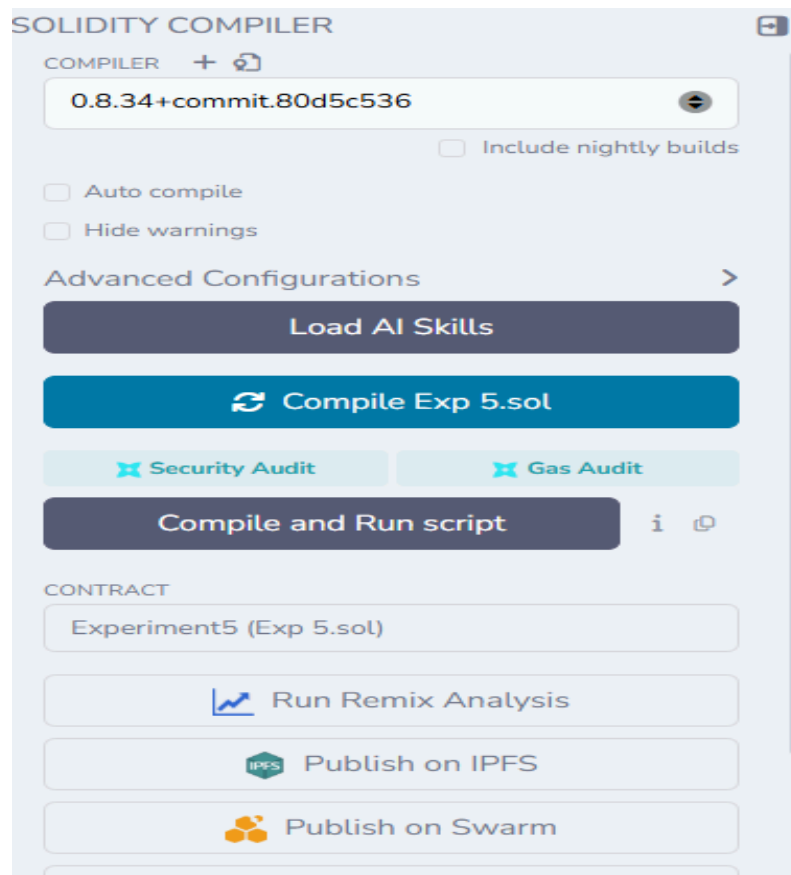
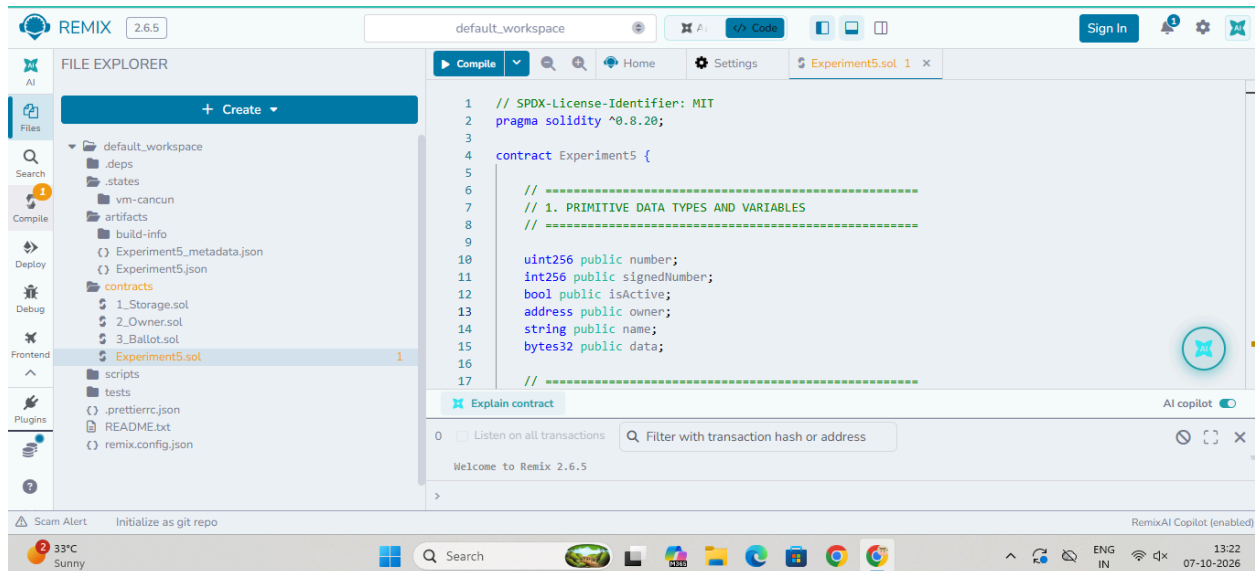
Read-only functions can be used to retrieve information, while state-changing functions are executed as transactions and consume gas.

Step 11: Complete the 19 Assignments

All 19 assignments provided in the LearnETH Solidity Basics Course are completed by following the same process of:



OUTPUT : -



REMIX

default_workspace

AI

Code

Home

Settings

Exp 5.sol

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancun

Account 1

0x5B3...eddC4

89.999 ETH

Deploy

Deployed Contracts 1

Transaction History 9

Deployed Contracts 1

Add Contract

Interact with a deployed contract

Experiment5

0xD7A_F771B

Remix VM

28min ago

Functions

+

Low Level interaction

-

calldata

Value

10

ether

Compile

Home

Settings

Exp 5.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract Experiment5 {
5
6     // =====
7     // 1. Primitive Data Types
8     // =====
9
10    uint public number = 10;
11    int public signedNumber = -5;
12    bool public isActive = true;
13
14    address public owner;
15    string public studentName = "Siddharth";
16    bytes32 public data = "Solidity";
17
18
19
20 }
```

Explain contract

AI copilot

0

☐ Listen on all transactions

Filter with transaction hash or address

command line interface

- Right-click on a JavaScript file in the file explorer and then click 'Run'

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

>

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancel

Account 1

0x5B3...eddC4

89.999 ETH

Deploy

Deployed Contracts 1

Transaction History 9

Deployed Contracts 1

+ Add Contract

Interact with a deployed contract

Experiment5

0xD7A...F771B

Remix VM

30min ago

Functions

+

Low level interaction

-

calldata

Value

10

ether

Compile

Home

Settings

Exp 5.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.20;
3
4 contract Experiment5 {
5
6     // =====
7     // 1. Primitive Data Types
8     // =====
9
10    uint public number = 10;
11    int public signedNumber = -5;
12    bool public isActive = true;
13
14    address public owner;
15    string public studentName = "Siddharth";
16    bytes32 public data = "Solidity";
17
18    // =====
19    // 2. Structs
20    // =====
21
22    struct Student {
23        string name;
24        uint age;
25    }
26
27    Student public student;
28
29    // =====
30    // 3. Arrays
31    // =====
32
33    uint[] public numbers;
34    Student[] public students;
35
36    // =====
37    // 4. Mapping
38    // =====
39
40    mapping(uint => Student) public studentMap;
41
42    // =====
43    // 5. Events
44    // =====
45
46    event StudentAdded(string name, uint age);
47
48    function addStudent(string memory name, uint age) public {
49        Student memory newStudent = Student(name, age);
50        studentMap[uint256(keccak256(name))] = newStudent;
51        emit StudentAdded(name, age);
52    }
53
54    // =====
55    // 6. Functions
56    // =====
57
58    function getNumber() public view returns (uint) {
59        return number;
60    }
61
62    function setNumber(uint _number) public {
63        number = _number;
64    }
65
66    function getSignedNumber() public view returns (int) {
67        return signedNumber;
68    }
69
70    function setSignedNumber(int _signedNumber) public {
71        signedNumber = _signedNumber;
72    }
73
74    function getIsActive() public view returns (bool) {
75        return isActive;
76    }
77
78    function setIsActive(bool _isActive) public {
79        isActive = _isActive;
80    }
81
82    function getOwner() public view returns (address) {
83        return owner;
84    }
85
86    function setOwner(address _owner) public {
87        owner = _owner;
88    }
89
90    function getStudentName() public view returns (string) {
91        return studentName;
92    }
93
94    function setStudentName(string memory _studentName) public {
95        studentName = _studentName;
96    }
97
98    function getData() public view returns (bytes32) {
99        return data;
100    }
101
102    function setData(bytes32 _data) public {
103        data = _data;
104    }
105
106    function getNumbers() public view returns (uint[]) {
107        return numbers;
108    }
109
110    function addNumber(uint _number) public {
111        numbers.push(_number);
112    }
113
114    function getStudents() public view returns (Student[]) {
115        return students;
116    }
117
118    function addStudentToArray(string memory name, uint age) public {
119        Student memory newStudent = Student(name, age);
120        students.push(newStudent);
121    }
122
123    function getStudentMap() public view returns (mapping(uint => Student)) {
124        return studentMap;
125    }
126
127    function addStudentToMap(string memory name, uint age) public {
128        Student memory newStudent = Student(name, age);
129        studentMap[uint256(keccak256(name))] = newStudent;
130    }
131
132    // =====
133    // 7. Errors
134    // =====
135
136    error InvalidNumber(uint number);
137    error InvalidSignedNumber(int signedNumber);
138    error InvalidIsActive(bool isActive);
139    error InvalidOwner(address owner);
140    error InvalidStudentName(string studentName);
141    error InvalidData(bytes32 data);
142    error InvalidNumbers(uint[] numbers);
143    error InvalidStudents(Student[] students);
144    error InvalidStudentMap(mapping(uint => Student) studentMap);
145
146    function validateNumber(uint number) public pure {
147        require(number > 0, InvalidNumber(number));
148    }
149
150    function validateSignedNumber(int signedNumber) public pure {
151        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
152    }
153
154    function validateIsActive(bool isActive) public pure {
155        require(isActive, InvalidIsActive(isActive));
156    }
157
158    function validateOwner(address owner) public pure {
159        require(owner != address(0), InvalidOwner(owner));
160    }
161
162    function validateStudentName(string memory studentName) public pure {
163        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
164    }
165
166    function validateData(bytes32 data) public pure {
167        require(data != bytes32(0), InvalidData(data));
168    }
169
170    function validateNumbers(uint[] memory numbers) public pure {
171        require(numbers.length > 0, InvalidNumbers(numbers));
172    }
173
174    function validateStudents(Student[] memory students) public pure {
175        require(students.length > 0, InvalidStudents(students));
176    }
177
178    function validateStudentMap(mapping(uint => Student) memory studentMap) public pure {
179        require(studentMap.length > 0, InvalidStudentMap(studentMap));
180    }
181
182    // =====
183    // 8. Miscellaneous
184    // =====
185
186    function getGasUsed() public view returns (uint) {
187        return gasleft();
188    }
189
190    function getBlockHash(uint blockNumber) public view returns (bytes32) {
191        return blockhash(blockNumber);
192    }
193
194    function getBlockNumber() public view returns (uint) {
195        return block.number;
196    }
197
198    function getContractAddress() public view returns (address) {
199        return address(this);
200    }
201
202    function getFrom() public view returns (address) {
203        return msg.sender;
204    }
205
206    function getTo() public view returns (address) {
207        return address(this);
208    }
209
210    function getTransactionCost() public view returns (uint) {
211        return msg.value;
212    }
213
214    // =====
215    // 9. Constants
216    // =====
217
218    uint constant MAX_NUMBER = 100;
219    int constant MIN_SIGNED_NUMBER = -100;
220    bool constant DEFAULT_IS_ACTIVE = false;
221    address constant DEFAULT_OWNER = address(0);
222    string constant DEFAULT_STUDENT_NAME = "";
223    bytes32 constant DEFAULT_DATA = bytes32(0);
224    uint[] constant DEFAULT_NUMBERS = [];
225    Student[] constant DEFAULT_STUDENTS = [];
226    mapping(uint => Student) constant DEFAULT_STUDENT_MAP = mapping(uint => Student)();
227
228    // =====
229    // 10. Private Functions
230    // =====
231
232    function _validateNumber(uint number) private pure {
233        require(number > 0, InvalidNumber(number));
234    }
235
236    function _validateSignedNumber(int signedNumber) private pure {
237        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
238    }
239
240    function _validateIsActive(bool isActive) private pure {
241        require(isActive, InvalidIsActive(isActive));
242    }
243
244    function _validateOwner(address owner) private pure {
245        require(owner != address(0), InvalidOwner(owner));
246    }
247
248    function _validateStudentName(string memory studentName) private pure {
249        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
250    }
251
252    function _validateData(bytes32 data) private pure {
253        require(data != bytes32(0), InvalidData(data));
254    }
255
256    function _validateNumbers(uint[] memory numbers) private pure {
257        require(numbers.length > 0, InvalidNumbers(numbers));
258    }
259
260    function _validateStudents(Student[] memory students) private pure {
261        require(students.length > 0, InvalidStudents(students));
262    }
263
264    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
265        require(studentMap.length > 0, InvalidStudentMap(studentMap));
266    }
267
268    // =====
269    // 11. Private Constants
270    // =====
271
272    uint private _MAX_NUMBER = 100;
273    int private _MIN_SIGNED_NUMBER = -100;
274    bool private _DEFAULT_IS_ACTIVE = false;
275    address private _DEFAULT_OWNER = address(0);
276    string private _DEFAULT_STUDENT_NAME = "";
277    bytes32 private _DEFAULT_DATA = bytes32(0);
278    uint[] private _DEFAULT_NUMBERS = [];
279    Student[] private _DEFAULT_STUDENTS = [];
280    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
281
282    // =====
283    // 12. Private Functions
284    // =====
285
286    function _validateNumber(uint number) private pure {
287        require(number > 0, InvalidNumber(number));
288    }
289
290    function _validateSignedNumber(int signedNumber) private pure {
291        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
292    }
293
294    function _validateIsActive(bool isActive) private pure {
295        require(isActive, InvalidIsActive(isActive));
296    }
297
298    function _validateOwner(address owner) private pure {
299        require(owner != address(0), InvalidOwner(owner));
300    }
301
302    function _validateStudentName(string memory studentName) private pure {
303        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
304    }
305
306    function _validateData(bytes32 data) private pure {
307        require(data != bytes32(0), InvalidData(data));
308    }
309
310    function _validateNumbers(uint[] memory numbers) private pure {
311        require(numbers.length > 0, InvalidNumbers(numbers));
312    }
313
314    function _validateStudents(Student[] memory students) private pure {
315        require(students.length > 0, InvalidStudents(students));
316    }
317
318    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
319        require(studentMap.length > 0, InvalidStudentMap(studentMap));
320    }
321
322    // =====
323    // 13. Private Constants
324    // =====
325
326    uint private _MAX_NUMBER = 100;
327    int private _MIN_SIGNED_NUMBER = -100;
328    bool private _DEFAULT_IS_ACTIVE = false;
329    address private _DEFAULT_OWNER = address(0);
330    string private _DEFAULT_STUDENT_NAME = "";
331    bytes32 private _DEFAULT_DATA = bytes32(0);
332    uint[] private _DEFAULT_NUMBERS = [];
333    Student[] private _DEFAULT_STUDENTS = [];
334    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
335
336    // =====
337    // 14. Private Functions
338    // =====
339
340    function _validateNumber(uint number) private pure {
341        require(number > 0, InvalidNumber(number));
342    }
343
344    function _validateSignedNumber(int signedNumber) private pure {
345        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
346    }
347
348    function _validateIsActive(bool isActive) private pure {
349        require(isActive, InvalidIsActive(isActive));
350    }
351
352    function _validateOwner(address owner) private pure {
353        require(owner != address(0), InvalidOwner(owner));
354    }
355
356    function _validateStudentName(string memory studentName) private pure {
357        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
358    }
359
360    function _validateData(bytes32 data) private pure {
361        require(data != bytes32(0), InvalidData(data));
362    }
363
364    function _validateNumbers(uint[] memory numbers) private pure {
365        require(numbers.length > 0, InvalidNumbers(numbers));
366    }
367
368    function _validateStudents(Student[] memory students) private pure {
369        require(students.length > 0, InvalidStudents(students));
370    }
371
372    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
373        require(studentMap.length > 0, InvalidStudentMap(studentMap));
374    }
375
376    // =====
377    // 15. Private Constants
378    // =====
379
380    uint private _MAX_NUMBER = 100;
381    int private _MIN_SIGNED_NUMBER = -100;
382    bool private _DEFAULT_IS_ACTIVE = false;
383    address private _DEFAULT_OWNER = address(0);
384    string private _DEFAULT_STUDENT_NAME = "";
385    bytes32 private _DEFAULT_DATA = bytes32(0);
386    uint[] private _DEFAULT_NUMBERS = [];
387    Student[] private _DEFAULT_STUDENTS = [];
388    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
389
390    // =====
391    // 16. Private Functions
392    // =====
393
394    function _validateNumber(uint number) private pure {
395        require(number > 0, InvalidNumber(number));
396    }
397
398    function _validateSignedNumber(int signedNumber) private pure {
399        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
400    }
401
402    function _validateIsActive(bool isActive) private pure {
403        require(isActive, InvalidIsActive(isActive));
404    }
405
406    function _validateOwner(address owner) private pure {
407        require(owner != address(0), InvalidOwner(owner));
408    }
409
410    function _validateStudentName(string memory studentName) private pure {
411        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
412    }
413
414    function _validateData(bytes32 data) private pure {
415        require(data != bytes32(0), InvalidData(data));
416    }
417
418    function _validateNumbers(uint[] memory numbers) private pure {
419        require(numbers.length > 0, InvalidNumbers(numbers));
420    }
421
422    function _validateStudents(Student[] memory students) private pure {
423        require(students.length > 0, InvalidStudents(students));
424    }
425
426    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
427        require(studentMap.length > 0, InvalidStudentMap(studentMap));
428    }
429
430    // =====
431    // 17. Private Constants
432    // =====
433
434    uint private _MAX_NUMBER = 100;
435    int private _MIN_SIGNED_NUMBER = -100;
436    bool private _DEFAULT_IS_ACTIVE = false;
437    address private _DEFAULT_OWNER = address(0);
438    string private _DEFAULT_STUDENT_NAME = "";
439    bytes32 private _DEFAULT_DATA = bytes32(0);
440    uint[] private _DEFAULT_NUMBERS = [];
441    Student[] private _DEFAULT_STUDENTS = [];
442    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
443
444    // =====
445    // 18. Private Functions
446    // =====
447
448    function _validateNumber(uint number) private pure {
449        require(number > 0, InvalidNumber(number));
450    }
451
452    function _validateSignedNumber(int signedNumber) private pure {
453        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
454    }
455
456    function _validateIsActive(bool isActive) private pure {
457        require(isActive, InvalidIsActive(isActive));
458    }
459
460    function _validateOwner(address owner) private pure {
461        require(owner != address(0), InvalidOwner(owner));
462    }
463
464    function _validateStudentName(string memory studentName) private pure {
465        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
466    }
467
468    function _validateData(bytes32 data) private pure {
469        require(data != bytes32(0), InvalidData(data));
470    }
471
472    function _validateNumbers(uint[] memory numbers) private pure {
473        require(numbers.length > 0, InvalidNumbers(numbers));
474    }
475
476    function _validateStudents(Student[] memory students) private pure {
477        require(students.length > 0, InvalidStudents(students));
478    }
479
480    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
481        require(studentMap.length > 0, InvalidStudentMap(studentMap));
482    }
483
484    // =====
485    // 19. Private Constants
486    // =====
487
488    uint private _MAX_NUMBER = 100;
489    int private _MIN_SIGNED_NUMBER = -100;
490    bool private _DEFAULT_IS_ACTIVE = false;
491    address private _DEFAULT_OWNER = address(0);
492    string private _DEFAULT_STUDENT_NAME = "";
493    bytes32 private _DEFAULT_DATA = bytes32(0);
494    uint[] private _DEFAULT_NUMBERS = [];
495    Student[] private _DEFAULT_STUDENTS = [];
496    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
497
498    // =====
499    // 20. Private Functions
500    // =====
501
502    function _validateNumber(uint number) private pure {
503        require(number > 0, InvalidNumber(number));
504    }
505
506    function _validateSignedNumber(int signedNumber) private pure {
507        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
508    }
509
510    function _validateIsActive(bool isActive) private pure {
511        require(isActive, InvalidIsActive(isActive));
512    }
513
514    function _validateOwner(address owner) private pure {
515        require(owner != address(0), InvalidOwner(owner));
516    }
517
518    function _validateStudentName(string memory studentName) private pure {
519        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
520    }
521
522    function _validateData(bytes32 data) private pure {
523        require(data != bytes32(0), InvalidData(data));
524    }
525
526    function _validateNumbers(uint[] memory numbers) private pure {
527        require(numbers.length > 0, InvalidNumbers(numbers));
528    }
529
530    function _validateStudents(Student[] memory students) private pure {
531        require(students.length > 0, InvalidStudents(students));
532    }
533
534    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
535        require(studentMap.length > 0, InvalidStudentMap(studentMap));
536    }
537
538    // =====
539    // 21. Private Constants
540    // =====
541
542    uint private _MAX_NUMBER = 100;
543    int private _MIN_SIGNED_NUMBER = -100;
544    bool private _DEFAULT_IS_ACTIVE = false;
545    address private _DEFAULT_OWNER = address(0);
546    string private _DEFAULT_STUDENT_NAME = "";
547    bytes32 private _DEFAULT_DATA = bytes32(0);
548    uint[] private _DEFAULT_NUMBERS = [];
549    Student[] private _DEFAULT_STUDENTS = [];
550    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
551
552    // =====
553    // 22. Private Functions
554    // =====
555
556    function _validateNumber(uint number) private pure {
557        require(number > 0, InvalidNumber(number));
558    }
559
560    function _validateSignedNumber(int signedNumber) private pure {
561        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
562    }
563
564    function _validateIsActive(bool isActive) private pure {
565        require(isActive, InvalidIsActive(isActive));
566    }
567
568    function _validateOwner(address owner) private pure {
569        require(owner != address(0), InvalidOwner(owner));
570    }
571
572    function _validateStudentName(string memory studentName) private pure {
573        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
574    }
575
576    function _validateData(bytes32 data) private pure {
577        require(data != bytes32(0), InvalidData(data));
578    }
579
580    function _validateNumbers(uint[] memory numbers) private pure {
581        require(numbers.length > 0, InvalidNumbers(numbers));
582    }
583
584    function _validateStudents(Student[] memory students) private pure {
585        require(students.length > 0, InvalidStudents(students));
586    }
587
588    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
589        require(studentMap.length > 0, InvalidStudentMap(studentMap));
590    }
591
592    // =====
593    // 23. Private Constants
594    // =====
595
596    uint private _MAX_NUMBER = 100;
597    int private _MIN_SIGNED_NUMBER = -100;
598    bool private _DEFAULT_IS_ACTIVE = false;
599    address private _DEFAULT_OWNER = address(0);
600    string private _DEFAULT_STUDENT_NAME = "";
601    bytes32 private _DEFAULT_DATA = bytes32(0);
602    uint[] private _DEFAULT_NUMBERS = [];
603    Student[] private _DEFAULT_STUDENTS = [];
604    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
605
606    // =====
607    // 24. Private Functions
608    // =====
609
610    function _validateNumber(uint number) private pure {
611        require(number > 0, InvalidNumber(number));
612    }
613
614    function _validateSignedNumber(int signedNumber) private pure {
615        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
616    }
617
618    function _validateIsActive(bool isActive) private pure {
619        require(isActive, InvalidIsActive(isActive));
620    }
621
622    function _validateOwner(address owner) private pure {
623        require(owner != address(0), InvalidOwner(owner));
624    }
625
626    function _validateStudentName(string memory studentName) private pure {
627        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
628    }
629
630    function _validateData(bytes32 data) private pure {
631        require(data != bytes32(0), InvalidData(data));
632    }
633
634    function _validateNumbers(uint[] memory numbers) private pure {
635        require(numbers.length > 0, InvalidNumbers(numbers));
636    }
637
638    function _validateStudents(Student[] memory students) private pure {
639        require(students.length > 0, InvalidStudents(students));
640    }
641
642    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
643        require(studentMap.length > 0, InvalidStudentMap(studentMap));
644    }
645
646    // =====
647    // 25. Private Constants
648    // =====
649
650    uint private _MAX_NUMBER = 100;
651    int private _MIN_SIGNED_NUMBER = -100;
652    bool private _DEFAULT_IS_ACTIVE = false;
653    address private _DEFAULT_OWNER = address(0);
654    string private _DEFAULT_STUDENT_NAME = "";
655    bytes32 private _DEFAULT_DATA = bytes32(0);
656    uint[] private _DEFAULT_NUMBERS = [];
657    Student[] private _DEFAULT_STUDENTS = [];
658    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
659
660    // =====
661    // 26. Private Functions
662    // =====
663
664    function _validateNumber(uint number) private pure {
665        require(number > 0, InvalidNumber(number));
666    }
667
668    function _validateSignedNumber(int signedNumber) private pure {
669        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
670    }
669
670    function _validateIsActive(bool isActive) private pure {
671        require(isActive, InvalidIsActive(isActive));
672    }
673
674    function _validateOwner(address owner) private pure {
675        require(owner != address(0), InvalidOwner(owner));
676    }
677
678    function _validateStudentName(string memory studentName) private pure {
679        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
680    }
681
682    function _validateData(bytes32 data) private pure {
683        require(data != bytes32(0), InvalidData(data));
684    }
685
686    function _validateNumbers(uint[] memory numbers) private pure {
687        require(numbers.length > 0, InvalidNumbers(numbers));
688    }
689
690    function _validateStudents(Student[] memory students) private pure {
691        require(students.length > 0, InvalidStudents(students));
692    }
693
694    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
695        require(studentMap.length > 0, InvalidStudentMap(studentMap));
696    }
697
698    // =====
699    // 27. Private Constants
700    // =====
701
702    uint private _MAX_NUMBER = 100;
703    int private _MIN_SIGNED_NUMBER = -100;
704    bool private _DEFAULT_IS_ACTIVE = false;
705    address private _DEFAULT_OWNER = address(0);
706    string private _DEFAULT_STUDENT_NAME = "";
707    bytes32 private _DEFAULT_DATA = bytes32(0);
708    uint[] private _DEFAULT_NUMBERS = [];
709    Student[] private _DEFAULT_STUDENTS = [];
710    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
711
712    // =====
713    // 28. Private Functions
714    // =====
715
716    function _validateNumber(uint number) private pure {
717        require(number > 0, InvalidNumber(number));
718    }
719
720    function _validateSignedNumber(int signedNumber) private pure {
721        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
722    }
723
724    function _validateIsActive(bool isActive) private pure {
725        require(isActive, InvalidIsActive(isActive));
726    }
727
728    function _validateOwner(address owner) private pure {
729        require(owner != address(0), InvalidOwner(owner));
730    }
731
732    function _validateStudentName(string memory studentName) private pure {
733        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
734    }
735
736    function _validateData(bytes32 data) private pure {
737        require(data != bytes32(0), InvalidData(data));
738    }
739
740    function _validateNumbers(uint[] memory numbers) private pure {
741        require(numbers.length > 0, InvalidNumbers(numbers));
742    }
743
744    function _validateStudents(Student[] memory students) private pure {
745        require(students.length > 0, InvalidStudents(students));
746    }
747
748    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
749        require(studentMap.length > 0, InvalidStudentMap(studentMap));
750    }
751
752    // =====
753    // 29. Private Constants
754    // =====
755
756    uint private _MAX_NUMBER = 100;
757    int private _MIN_SIGNED_NUMBER = -100;
758    bool private _DEFAULT_IS_ACTIVE = false;
759    address private _DEFAULT_OWNER = address(0);
760    string private _DEFAULT_STUDENT_NAME = "";
761    bytes32 private _DEFAULT_DATA = bytes32(0);
762    uint[] private _DEFAULT_NUMBERS = [];
763    Student[] private _DEFAULT_STUDENTS = [];
764    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
765
766    // =====
767    // 30. Private Functions
768    // =====
769
770    function _validateNumber(uint number) private pure {
771        require(number > 0, InvalidNumber(number));
772    }
773
774    function _validateSignedNumber(int signedNumber) private pure {
775        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
776    }
777
778    function _validateIsActive(bool isActive) private pure {
779        require(isActive, InvalidIsActive(isActive));
780    }
781
782    function _validateOwner(address owner) private pure {
783        require(owner != address(0), InvalidOwner(owner));
784    }
785
786    function _validateStudentName(string memory studentName) private pure {
787        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
788    }
789
790    function _validateData(bytes32 data) private pure {
791        require(data != bytes32(0), InvalidData(data));
792    }
793
794    function _validateNumbers(uint[] memory numbers) private pure {
795        require(numbers.length > 0, InvalidNumbers(numbers));
796    }
797
798    function _validateStudents(Student[] memory students) private pure {
799        require(students.length > 0, InvalidStudents(students));
800    }
801
802    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
803        require(studentMap.length > 0, InvalidStudentMap(studentMap));
804    }
805
806    // =====
807    // 31. Private Constants
808    // =====
809
810    uint private _MAX_NUMBER = 100;
811    int private _MIN_SIGNED_NUMBER = -100;
812    bool private _DEFAULT_IS_ACTIVE = false;
813    address private _DEFAULT_OWNER = address(0);
814    string private _DEFAULT_STUDENT_NAME = "";
815    bytes32 private _DEFAULT_DATA = bytes32(0);
816    uint[] private _DEFAULT_NUMBERS = [];
817    Student[] private _DEFAULT_STUDENTS = [];
818    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
819
820    // =====
821    // 32. Private Functions
822    // =====
823
824    function _validateNumber(uint number) private pure {
825        require(number > 0, InvalidNumber(number));
826    }
827
828    function _validateSignedNumber(int signedNumber) private pure {
829        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
830    }
829
830    function _validateIsActive(bool isActive) private pure {
831        require(isActive, InvalidIsActive(isActive));
832    }
833
834    function _validateOwner(address owner) private pure {
835        require(owner != address(0), InvalidOwner(owner));
836    }
837
838    function _validateStudentName(string memory studentName) private pure {
839        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
840    }
841
842    function _validateData(bytes32 data) private pure {
843        require(data != bytes32(0), InvalidData(data));
844    }
845
846    function _validateNumbers(uint[] memory numbers) private pure {
847        require(numbers.length > 0, InvalidNumbers(numbers));
848    }
849
850    function _validateStudents(Student[] memory students) private pure {
851        require(students.length > 0, InvalidStudents(students));
852    }
853
854    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
855        require(studentMap.length > 0, InvalidStudentMap(studentMap));
856    }
857
858    // =====
859    // 33. Private Constants
860    // =====
861
862    uint private _MAX_NUMBER = 100;
863    int private _MIN_SIGNED_NUMBER = -100;
864    bool private _DEFAULT_IS_ACTIVE = false;
865    address private _DEFAULT_OWNER = address(0);
866    string private _DEFAULT_STUDENT_NAME = "";
867    bytes32 private _DEFAULT_DATA = bytes32(0);
868    uint[] private _DEFAULT_NUMBERS = [];
869    Student[] private _DEFAULT_STUDENTS = [];
870    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
871
872    // =====
873    // 34. Private Functions
874    // =====
875
876    function _validateNumber(uint number) private pure {
877        require(number > 0, InvalidNumber(number));
878    }
879
880    function _validateSignedNumber(int signedNumber) private pure {
881        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
882    }
883
884    function _validateIsActive(bool isActive) private pure {
885        require(isActive, InvalidIsActive(isActive));
886    }
887
888    function _validateOwner(address owner) private pure {
889        require(owner != address(0), InvalidOwner(owner));
890    }
891
892    function _validateStudentName(string memory studentName) private pure {
893        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
894    }
895
896    function _validateData(bytes32 data) private pure {
897        require(data != bytes32(0), InvalidData(data));
898    }
899
900    function _validateNumbers(uint[] memory numbers) private pure {
901        require(numbers.length > 0, InvalidNumbers(numbers));
902    }
903
904    function _validateStudents(Student[] memory students) private pure {
905        require(students.length > 0, InvalidStudents(students));
906    }
907
908    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
909        require(studentMap.length > 0, InvalidStudentMap(studentMap));
910    }
911
912    // =====
913    // 35. Private Constants
914    // =====
915
916    uint private _MAX_NUMBER = 100;
917    int private _MIN_SIGNED_NUMBER = -100;
918    bool private _DEFAULT_IS_ACTIVE = false;
919    address private _DEFAULT_OWNER = address(0);
920    string private _DEFAULT_STUDENT_NAME = "";
921    bytes32 private _DEFAULT_DATA = bytes32(0);
922    uint[] private _DEFAULT_NUMBERS = [];
923    Student[] private _DEFAULT_STUDENTS = [];
924    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
925
926    // =====
927    // 36. Private Functions
928    // =====
929
930    function _validateNumber(uint number) private pure {
931        require(number > 0, InvalidNumber(number));
932    }
933
934    function _validateSignedNumber(int signedNumber) private pure {
935        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
936    }
937
938    function _validateIsActive(bool isActive) private pure {
939        require(isActive, InvalidIsActive(isActive));
940    }
941
942    function _validateOwner(address owner) private pure {
943        require(owner != address(0), InvalidOwner(owner));
944    }
945
946    function _validateStudentName(string memory studentName) private pure {
947        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
948    }
949
950    function _validateData(bytes32 data) private pure {
951        require(data != bytes32(0), InvalidData(data));
952    }
953
954    function _validateNumbers(uint[] memory numbers) private pure {
955        require(numbers.length > 0, InvalidNumbers(numbers));
956    }
957
958    function _validateStudents(Student[] memory students) private pure {
959        require(students.length > 0, InvalidStudents(students));
960    }
961
962    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
963        require(studentMap.length > 0, InvalidStudentMap(studentMap));
964    }
965
966    // =====
967    // 37. Private Constants
968    // =====
969
970    uint private _MAX_NUMBER = 100;
971    int private _MIN_SIGNED_NUMBER = -100;
972    bool private _DEFAULT_IS_ACTIVE = false;
973    address private _DEFAULT_OWNER = address(0);
974    string private _DEFAULT_STUDENT_NAME = "";
975    bytes32 private _DEFAULT_DATA = bytes32(0);
976    uint[] private _DEFAULT_NUMBERS = [];
977    Student[] private _DEFAULT_STUDENTS = [];
978    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
979
980    // =====
981    // 38. Private Functions
982    // =====
983
984    function _validateNumber(uint number) private pure {
985        require(number > 0, InvalidNumber(number));
986    }
987
988    function _validateSignedNumber(int signedNumber) private pure {
989        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
990    }
989
990    function _validateIsActive(bool isActive) private pure {
991        require(isActive, InvalidIsActive(isActive));
992    }
993
994    function _validateOwner(address owner) private pure {
995        require(owner != address(0), InvalidOwner(owner));
996    }
997
998    function _validateStudentName(string memory studentName) private pure {
999        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
1000    }
1001
1002    function _validateData(bytes32 data) private pure {
1003        require(data != bytes32(0), InvalidData(data));
1004    }
1005
1006    function _validateNumbers(uint[] memory numbers) private pure {
1007        require(numbers.length > 0, InvalidNumbers(numbers));
1008    }
1009
1010    function _validateStudents(Student[] memory students) private pure {
1011        require(students.length > 0, InvalidStudents(students));
1012    }
1013
1014    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
1015        require(studentMap.length > 0, InvalidStudentMap(studentMap));
1016    }
1017
1018    // =====
1019    // 39. Private Constants
1020    // =====
1021
1022    uint private _MAX_NUMBER = 100;
1023    int private _MIN_SIGNED_NUMBER = -100;
1024    bool private _DEFAULT_IS_ACTIVE = false;
1025    address private _DEFAULT_OWNER = address(0);
1026    string private _DEFAULT_STUDENT_NAME = "";
1027    bytes32 private _DEFAULT_DATA = bytes32(0);
1028    uint[] private _DEFAULT_NUMBERS = [];
1029    Student[] private _DEFAULT_STUDENTS = [];
1030    mapping(uint => Student) private _DEFAULT_STUDENT_MAP = mapping(uint => Student)();
1031
1032    // =====
1033    // 40. Private Functions
1034    // =====
1035
1036    function _validateNumber(uint number) private pure {
1037        require(number > 0, InvalidNumber(number));
1038    }
1039
1040    function _validateSignedNumber(int signedNumber) private pure {
1041        require(signedNumber > -100, InvalidSignedNumber(signedNumber));
1042    }
1043
1044    function _validateIsActive(bool isActive) private pure {
1045        require(isActive, InvalidIsActive(isActive));
1046    }
1047
1048    function _validateOwner(address owner) private pure {
1049        require(owner != address(0), InvalidOwner(owner));
1050    }
1051
1052    function _validateStudentName(string memory studentName) private pure {
1053        require(bytes(studentName).length > 0, InvalidStudentName(studentName));
1054    }
1055
1056    function _validateData(bytes32 data) private pure {
1057        require(data != bytes32(0), InvalidData(data));
1058    }
1059
1060    function _validateNumbers(uint[] memory numbers) private pure {
1061        require(numbers.length > 0, InvalidNumbers(numbers));
1062    }
1063
1064    function _validateStudents(Student[] memory students) private pure {
1065        require(students.length > 0, InvalidStudents(students));
1066    }
1067
1068    function _validateStudentMap(mapping(uint => Student) memory studentMap) private pure {
1069        require(studentMap.length > 0, Invalid
```

Low level interaction

callldata

Value

10

ether

Gas limit

auto

Transact

wei

gwei

finney

ether

Environment

Fork

Reset

Remix VM

Cancun

Account 1

0x5B3...eddC4

89.999 ETH

Experiment5 tx: 0x5db...fbb0a	deploy 35min ago
Experiment5 tx: 0xe0d...4a3cd	deploy 35min ago
Experiment5 tx: 0xeb6...598ed	deploy 37min ago
Experiment5 tx: 0x006...fe525	deploy 37min ago
Experiment5 tx: 0x4d6...dd527	deploy 41min ago

receive
tx: 0x1db...db5a2

transact
19min ago

depositBalance
tx: 0xbc0...f69c8

transact
30min ago

addToArray
tx: 0x636...ca023

transact
32min ago

addToArray
tx: 0x4bb...70413

transact
32min ago

addToArray
tx: 0x95f...31ce5

transact
32min ago

number

Call

0:uint256: 10

REMIX

default_workspace

Code

AI

DEPLOY & RUN TRANSACTIONS

Environment

Remix VM

Account 1

0x5B3...eddC4

89.999 ETH

Deploy

Deployed Contracts

Transaction History

param0 (uint256)

Calldata

Params

Auto-Fill with AI Samples

owner

Call

0.address: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

signedNumber

Call

status

Call

student

Call

studentName

Call

Compile

Home

Settings

Exp 5.sol

1 // SPDX-License-Identifier: MIT

2 pragma solidity ^0.8.20;

3

4 contract Experiment5 {

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

Scam Alert

Initialize as git repo

●

studentName

Call

0:string: Siddharth

REMIX

default_workspace

AI

Code

Home

Settings

Exp 5.sol

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancun

Account 1

0x5B3...eddC4

89.999 ETH

Deploy

Deployed Contracts

Transaction History

signedNumber

Call

status

Call

student

Call

studentName

Call

0:string: Siddharth

Low level interaction

Value

0

wei

Gas limit

auto

0

Compiled

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

mapping(address => uint) public balances;

// =====

// 4. Struct

// =====

struct Student {

string name;

uint age;

bool registered;

}

Student public student;

// =====

// 5. Enum

// =====

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

hash: 0x1f7...d6e14

Debug

CAL

[call] from: 0x5B38Da6a701c56854dCfc803Fc8875F56beddC4 to: Experiment5.studentName()

data: 0xb50...8ba31

Debug

REMIX

default_workspace

AI

Code

Home

Settings

Exp 5.sol

DEPLOY & RUN TRANSACTIONS

Environment

Fork

Reset

Remix VM

Cancun

Account 1

0x5B3...eddC4

89.999 ETH

Deploy

Deployed Contracts

Transaction History

numbers (uint256)

Call

param0 (uint256)

Calldata

Params

Auto-Fill with AI Samples

owner

Call

0:address: 0x5B38Da6a701c56854dCfc803Fc8875F56beddC4

signedNumber

Call

status

Call

student

Call

Compiled

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

contract Experiment5 {

// =====

// 1. Primitive Data Types

// =====

uint public number = 10;

int public signedNumber = -5;

bool public isActive = true;

address public owner;

string public studentName = "Siddharth";

bytes32 public data = "Solidity";

// =====

AI copilot

0

Listen on all transactions

Filter with transaction hash or address

data: 0xb50...8ba31

Debug

CAL

[call] from: 0x5B38Da6a701c56854dCfc803Fc8875F56beddC4 to: Experiment5.owner()

data: 0x8da...5cb5b

Debug